



OPERATING SYSTEM DESIGN

S Thenmozhi

Department of Computer Applications

OPERATING SYSTEM DESIGN

Storage Management & System Security

S Thenmozhi

Department of Computer Applications

Input/Output (I/O) operations are fundamental to the functioning of any computer system.

In an Operating System (OS), I/O operations involve communication between the CPU, memory, and external devices such as keyboard, mouse, disk, and network.

The efficiency and management of I/O operations significantly impact the overall performance and responsiveness of the system.

Importance of I/O Operations

- **System Interaction:** Enables interaction between user programs and the OS, allowing users to input data, receive output, and interact with applications.
- **Resource Management:** Responsible for managing resources and coordinating I/O requests from multiple processes.
- **Performance Optimization:** minimizing latency and improving overall responsiveness.

Key Concepts

1. **System Calls:** Mechanisms through which user programs request OS services for performing I/O operations.
2. **Buffering:** Technique used to enhance I/O performance by temporarily storing data in memory before it is transferred between devices and the CPU.

A system call is a method for a computer program to request a service from the kernel of the OS on which it is running.

There are variety of system calls: Process Control, File manipulation, Device management, Information maintenance, Communication and Protection

File Manipulation system calls: `open()`, `read()`, `write()`, `close()`

Types of I/O operations

Input operations: Receive data from external sources (e.g., keyboards, mice, sensors) and transferring it into the computer system's memory for processing.

Output operations: send data from the computer system's memory to external destinations (e.g., displays, printers, network interfaces) for presentation or storage.

Key characteristics

Direction:

Input operations: transfer data into the system,

output operations: transfer data out of the system.

Data Flow:

Input operations: user or external-generated data

output operations: system-generated or processed data.

Common I/O System Calls

Open():

Purpose: Opens a file or device and returns a file descriptor for subsequent operations.

Parameters: Filename or device name, access mode, permissions.

Example: `int fd = open("file.txt", O_RDONLY);`

Close():

Purpose: Closes a file descriptor and releases associated resources

Parameters: File descriptor.

Example: `close(fd);`

read():

Purpose: Reads data from a file descriptor into a buffer.

Parameters: File descriptor, buffer address, number of bytes to read.

Example: `read(fd, buffer, nbytes);`

write():

Purpose: Writes data from a buffer to a file descriptor

Parameters: File descriptor, buffer address, number of bytes to write

Example: `write(fd, buffer, nbytes);`

seek():

Purpose: Moves the file pointer associated with a file descriptor to a specified position

Parameters: File descriptor, offset, reference point.

Example: `lseek(fd, offset, SEEK_SET);`

Buffering

Buffering is a technique used to enhance the performance of input and output operations by temporarily storing data in memory before transferring it between devices and the CPU.

Purpose of Buffering

- Minimize the number of I/O system calls used
- Optimize the data transfer
- Smooth resource utilization

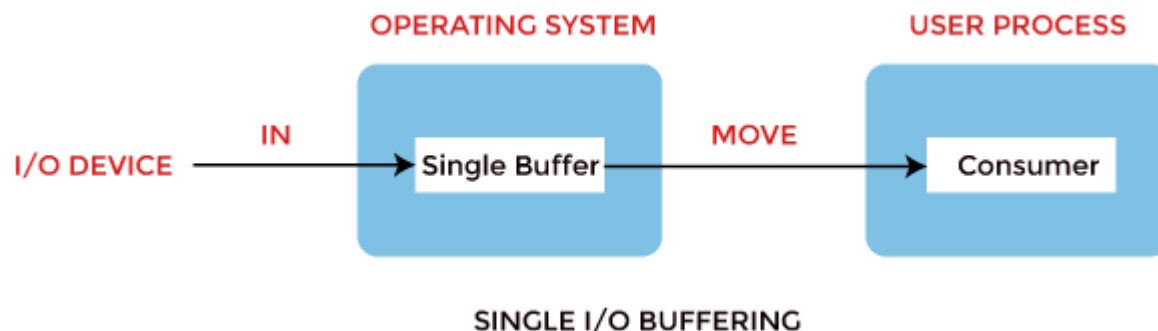
Types of Buffering

- Full Buffering – Entire data buffered in memory
- Line Buffering – Data buffered until newline character
- No Buffering – Without any intermediate buffering

Strategies of Buffering : Single, Double, Circular buffering

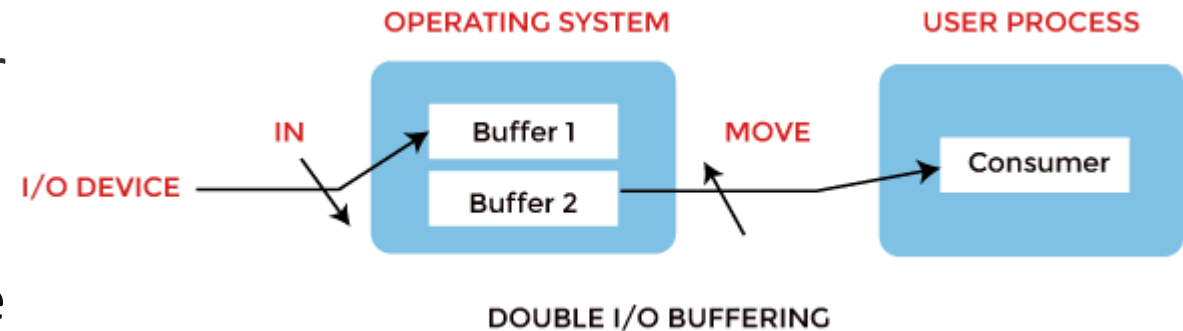
Single Buffering

- Only one buffer is used to transfer the data between two devices
- The producer produces one block of data into the buffer. After that, the consumer consumes the buffer.
- Line buffering or no buffering can be used for single buffering



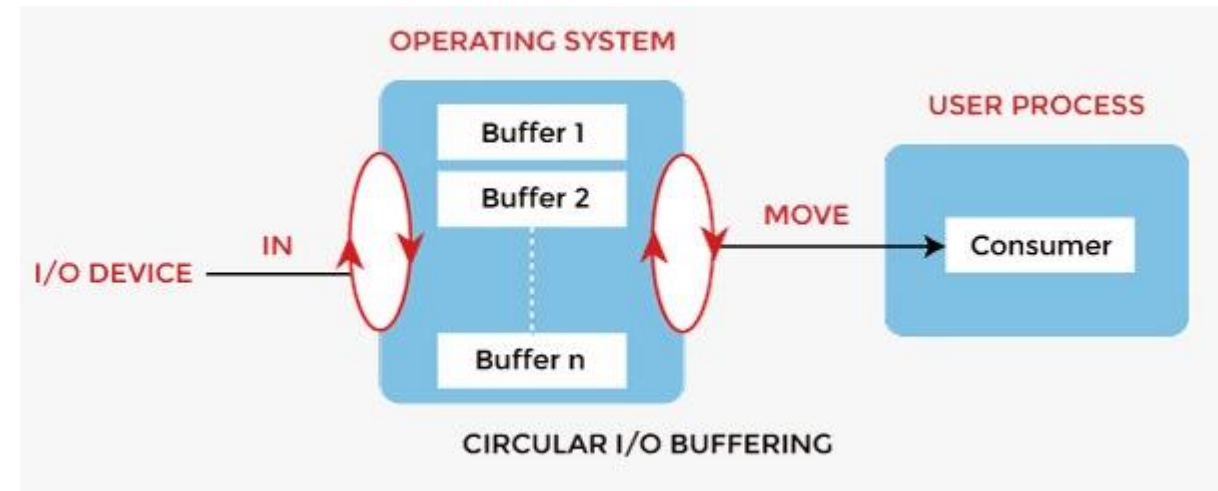
Double Buffering

- Two buffers are used
- The producer produces one buffer while the consumer consumes another buffer simultaneously
- Full buffering method can be used for double buffering
- Can be used Stream/Block devices



Circular Buffering

- More than two buffers are used
- The data do not directly pass from the producer to the consumer because the data would change due to overwriting of buffers before consumed.
- The producer can only fill up to buffer $x-1$ while data in buffer x is waiting to be consumed





THANK YOU

S Thenmozhi

Department of Computer Applications

thenmozhis@pes.edu

+91 80 6666 3333 Extn 393